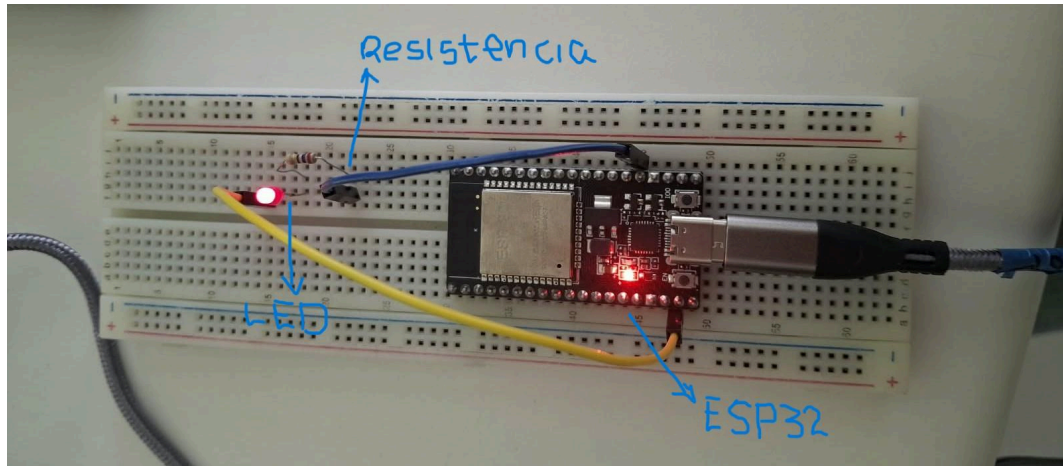


Taller 7

Ejercicio 1 — Encender y apagar un LED por Bluetooth clásico

Objetivo. Controlar un LED conectado al ESP32 desde una app Android usando Bluetooth clásico. Este ejercicio introduce ListPicker, BluetoothClient, envío de texto y retroalimentación de conexión.

- Circuito armado:



- Código Arduino:

```
tallerIoT | Arduino IDE 2.3.10
File Edit Sketch Tools Help

ESP32 Dev Module

tallerIoT.ino
10 pinMode(LED_PIN, OUTPUT);
11 digitalWrite(LED_PIN, LOW);
12
13 BT.begin("ESP32_LED");
14
15 Serial.println("Bluetooth started: ESP32_LED");
16
17
18 void loop() {
19   if (BT.available()) {
20     char c = BT.read();
21
22     if (c == '1') {
23       digitalWrite(LED_PIN, HIGH);
24       Serial.println("LED ON");
25     }
26
27     if (c == '0') {
28       digitalWrite(LED_PIN, LOW);
29       Serial.println("LED OFF");
30     }
31   }
32 }
33

Output Serial Monitor
Writing at 0x00102522 [=====] 93.0% 638976/687070 bytes...
Writing at 0x00107f36 [=====] 95.4% 655360/687070 bytes...
Writing at 0x0010dac1 [=====] 97.8% 671744/687070 bytes...
Writing at 0x001132a0 [=====] 100.0% 687070/687070 bytes...
Wrote 1061536 bytes (687070 compressed) at 0x00010000 in 9.5 seconds (889.6 kbit/s).
Verifying written data...
Hash of data verified.
Hard resetting via RTS pin...

Ln 33, Col 1 ESP32 Dev Module on COM5 09:35 a.m. 02/10/2026
```

- Bloques de App inventor:

```

when btnConectar .BeforePicking
do
  set btnConectar . Elements to BluetoothClient1 . AddressesAndNames

```

```

when btnConectar .AfterPicking
do
  if call BluetoothClient1 .Connect
    address btnConectar . Selection
  then set Label1 . Text to "CONNECTED"
  else set Label1 . Text to "ERROR"

```

```

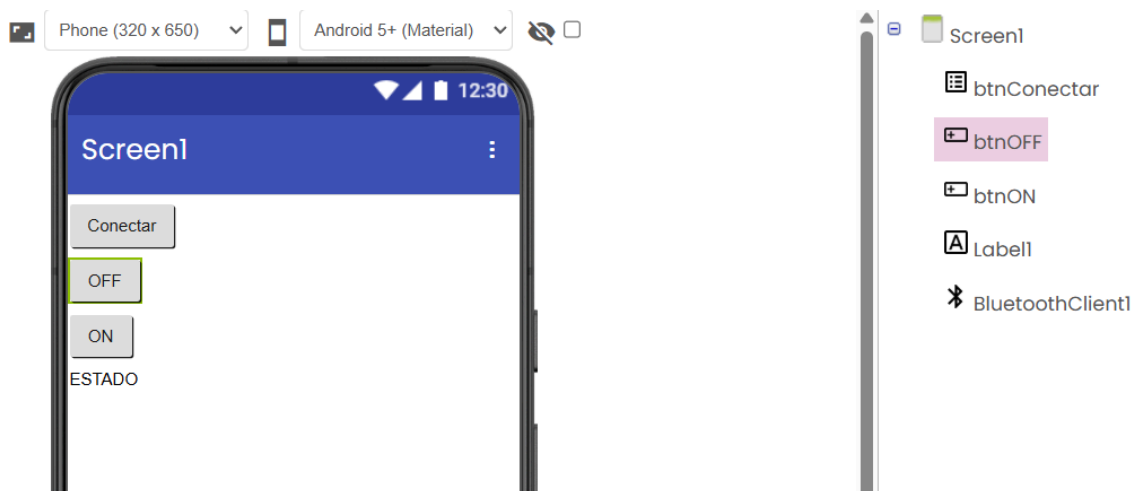
when btnON .Click
do
  call BluetoothClient1 .SendText
  text "1"

```

```

when btnOFF .Click
do
  call BluetoothClient1 .SendText
  text "0"

```

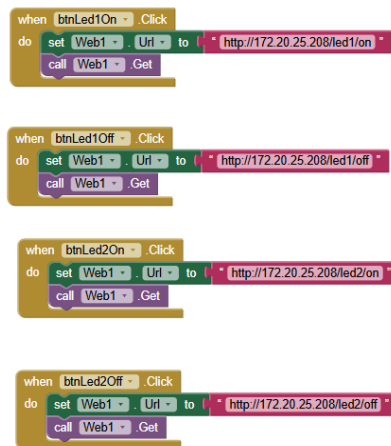
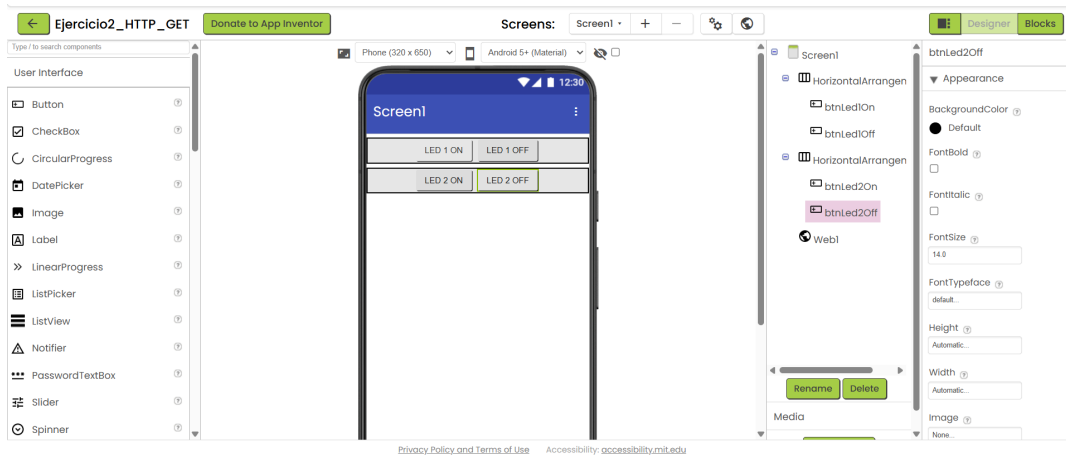


- Evidencia videográfica:

Ejercicio 2 — Control de dos LEDs por Wi-Fi usando HTTP GET

Objetivo. Crear un servidor web en el ESP32 y controlar dos salidas digitales mediante cuatro rutas HTTP GET.

- Circuito armado:



- Evidencia videográfica:

https://drive.google.com/file/d/1o-k9WORf4tk1nZC518qgCHjVsZafxasK/view?usp=drive_link

Ejercicio 3 — Control de un servomotor por Wi-Fi usando HTTP POST + JSON

Objetivo. Enviar desde un Slider un valor de 0 a 180, empaquetarlo como JSON, recibirlo en el ESP32 y mover un servo.

- Captura del programa Arduino

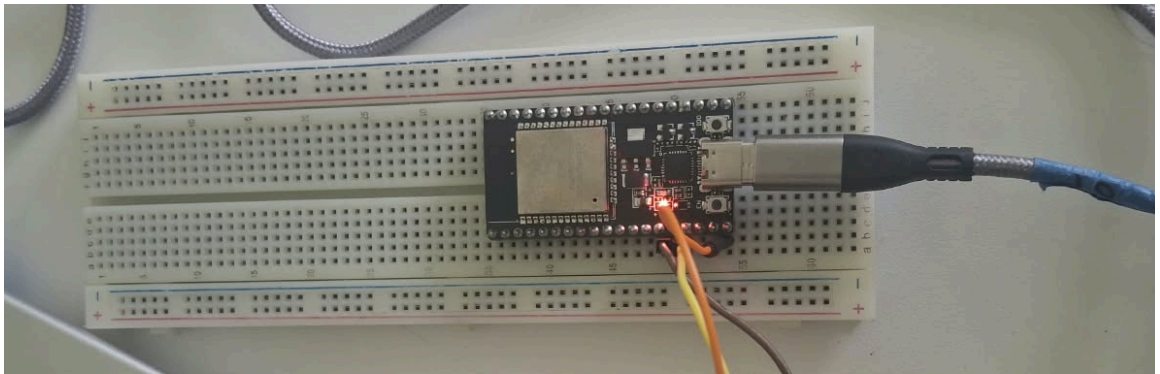
```

File Edit Sketch Tools Help
ESP32 Dev Module

xdd.ino
1 #include <WiFi.h>
2 #include <WebServer.h>
3 #include <ArduinoJson.h>
4 #include <ESP32Servo.h>
5
6 const char* ssid = "UPCH_CENTRAL";
7 const char* password = "CAVETANO2022";
8
9 WebServer server(80);
10
11 Servo myServo;
12 int angle = 90;
13
14 void handlePost() {
15   String recibido = server.arg("plain");
16
17   Serial.println("-----");
18   Serial.print("texto recibido: ");
19   Serial.println(recibido);
20
21   StaticJsonDocument<64> doc;
22   DeserializationError err = deserializeJson(doc, recibido);
23
24   StaticJsonDocument<96> res;
25
26   Output Serial Monitor X
27
28   Message (Enter to send message to 'ESP32 Dev Module' on 'COM5')
29   New Line 115200 baud
30
31   WiFi conectado
32   IP del ESP32: 172.20.26.69
33   Servidor HTTP iniciado

```

- Foto del servo conectado al ESP32.



- Captura de todos los bloques de App Inventor.

```

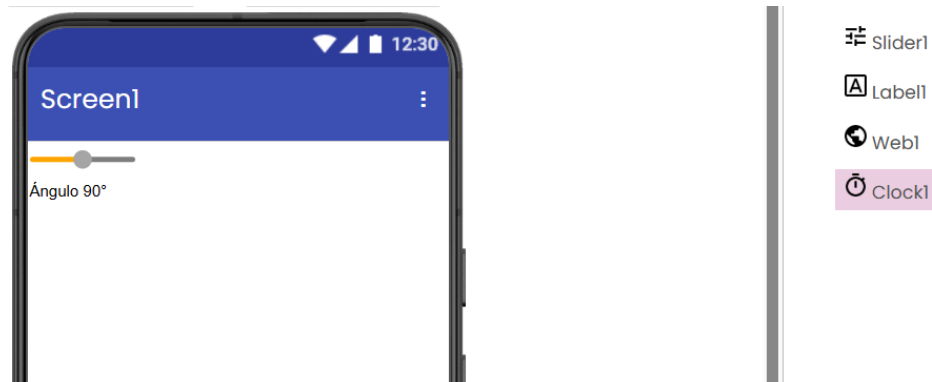
initialize global Objetivo to 90

when Slider1 .PositionChanged
  thumbPosition
  do
    set Label1 .Text to join ["Ángulo: ", get thumbPosition]
    set global Objetivo to get thumbPosition
    set Clock1 .TimerEnabled to true

when Web1 .GotText
  url responseCode responseType responseContent
  do
    set Label1 .Text to get responseContent

when Clock1 .Timer
  do
    set Clock1 .TimerEnabled to false
    set Web1 .Uri to "http://172.20.26.69/servo"
    set Web1 .RequestHeaders to make a list ["Content-Type: ", "application/json"]
    call Web1 .PostText
    text join [{"angulo": ", get global Objetivo ", "}]

```



- Captura del Monitor Serial mostrando un JSON recibido y el ángulo interpretado.

```

1  #include <ESP32Servo.h>
2
3  const char* ssid = "UPCH_CENTRAL";
4  const char* password = "CAYETANO2022";
5
6  WebServer server(80);
7
8  Servo myServo;
9  int angle = 90;
10
11 void handlePost() {
12   String recibido = server.arg("plain");
13   Serial.println("-----");
14   Serial.print("Texto recibido: ");
15   Serial.println(recibido);
16
17   StaticJsonDocument<64> doc;
18   DeserializationError err = deserializeJson(doc, recibido);
19   StaticJsonDocument<96> res;
20
21   if (!err && doc.containsKey("angulo")) {
22     angle = constrain(doc["angulo"], 0, 180);
23   }
24 }
25
26 void setup() {
27   Serial.begin(115200);
28   server.on("/setAngle", HTTP_POST, handlePost);
29   server.begin();
30   myServo.attach(9);
31 }
32
33 void loop() {
34   myServo.write(angle);
35   delay(1000);
36 }

```

Output Serial Monitor X

```

Message (Enter to send message to 'ESP32 Dev Module' on 'COM5')
-----
Texto recibido: {"angulo":85.814}
Angulo interpretado: 85
Respuesta enviada: {"ok":true,"angulo":85}
-----
Texto recibido: {"angulo":87.907}
Angulo interpretado: 87
Respuesta enviada: {"ok":true,"angulo":87}
-----
Texto recibido: {"angulo":88.1395}
Angulo interpretado: 88
Respuesta enviada: {"ok":true,"angulo":88}

```

- Demostración en vivo o video corto moviendo el Slider y observando el servo.

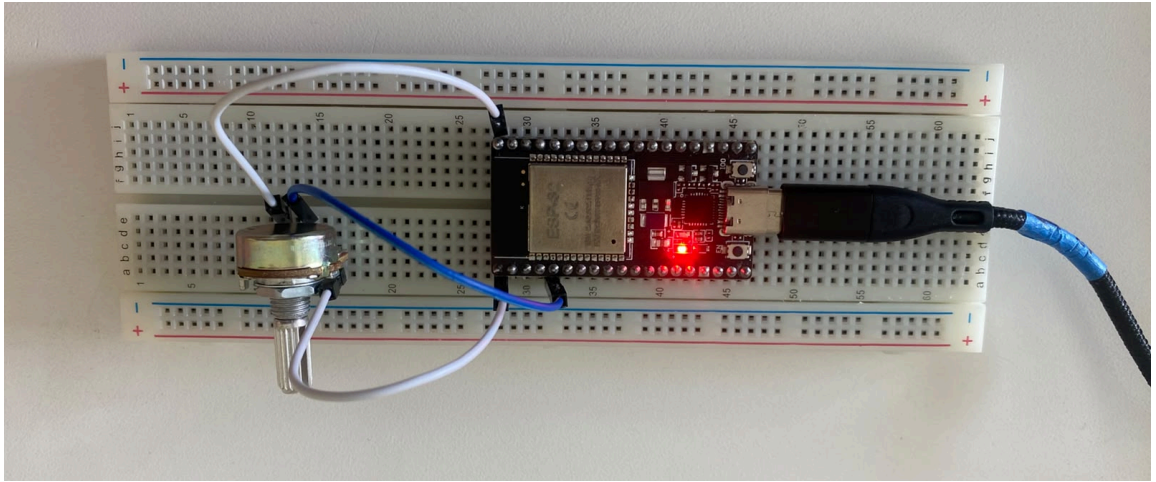
<https://drive.google.com/file/d/1dFbj9l0VSc05QdhvFqK30m25ruCf-0i0/view?usp=sharing>

Ejercicio 4 — Adquisición analógica en tiempo real: potenciómetro → flex sensor

Estructura. Este ejercicio tiene dos etapas: 4A es una práctica guiada con potenciómetro para validar ADC + HTTP GET; 4B es la adaptación asíncrona calificable con flex sensor, calibración y visualización.

4A. Práctica guiada con potenciómetro

- Circuito armado:



- Captura del Monitor Serial mostrando variación del ADC

```
sketch_001a.ino
1 const int PIN_ADC = 34;
2
3 void setup() {
4   Serial.begin(115200);
5 }
6
7 void loop() {
8   int valor = analogRead(PIN_ADC);
9   Serial.println(valor);
10  delay(200);
11 }
12
13 }
```

Output: Serial Monitor X

Message (Enter to send message to 'ESP32 Dev Module' on 'COM6')

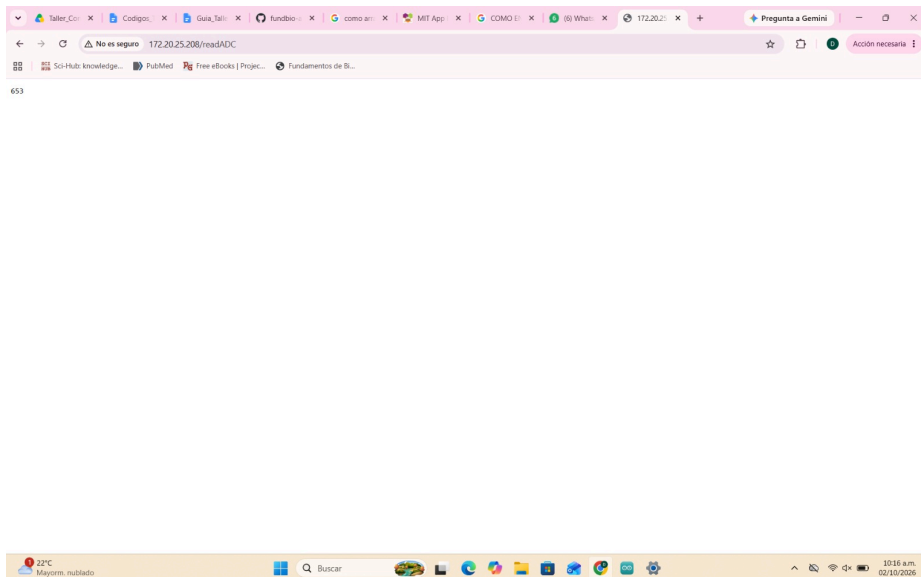
No Line Ending 115200 baud

10:00:06.697 -> 2177
10:00:06.927 -> 2178
10:00:06.114 -> 2176
10:00:06.809 -> 2178
10:00:06.496 -> 2177
10:00:06.718 -> 2176
10:00:06.901 -> 2178
10:00:07.100 -> 2180
10:00:07.311 -> 2181
10:00:07.512 -> 1840
10:00:07.697 -> 1883
10:00:07.908 -> 1927
10:00:08.111 -> 1108
10:00:08.317 -> 938
10:00:08.493 -> 764
10:00:08.715 -> 686
10:00:08.893 -> 496
10:00:09.113 -> 371
10:00:09.300 -> 272
10:00:09.493 -> 192

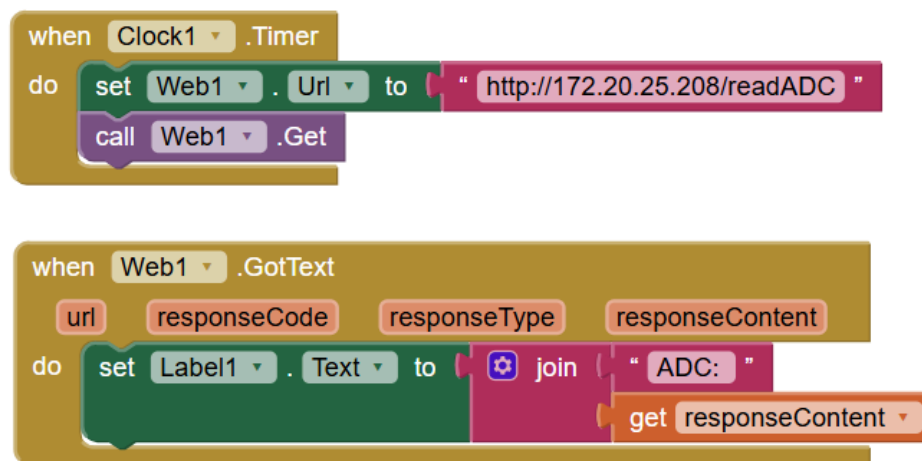
Evidencia videográfica:

https://drive.google.com/file/d/1Y7I5tURGz68QVtY6bmXyAjch1UhD0CwD/view?usp=drive_link

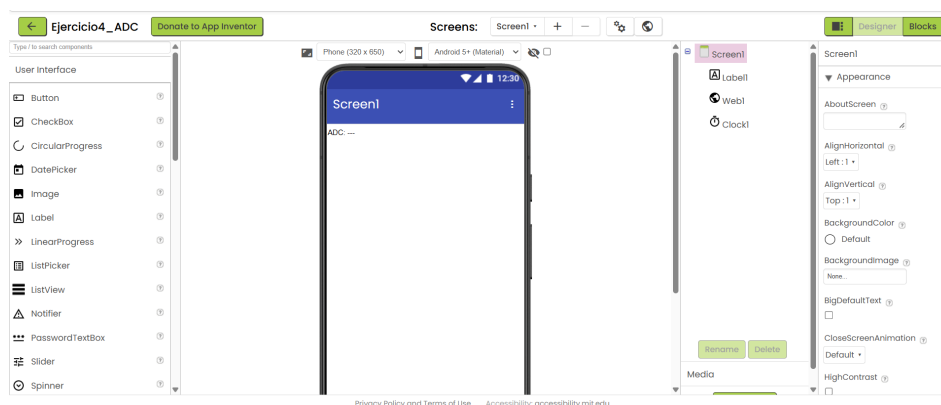
- Captura del navegador en /readADC.



- Captura de los bloques Clock1.Timer y Web1.GotText.



- Captura de screen del MIT App Inventor



- Evidencia videográfica:
https://drive.google.com/file/d/1gQsY7_WnHprihQ2ruhyVosMmL4hP6Fn9/view?usp=drive_link